

Software Design Document

Parallel Image Filtering System

Versiunea 1.0

Tipărit: 20 Aprilie 2026

Echipa PIF:

Bîrdeanu Aaron-Sasha
Băloi Raul Adrian Theodor
Cotrău Darius Andrei
Bărbascu Raul Andrei

Contents

1	Introducere	2
1.1	Scop	2
1.2	Domeniu de Aplicare	2
1.3	Definiții, Acronime și Abrevieri	2
1.4	Referințe	2
2	Prezentare Generală a Sistemului	3
3	Componentele Sistemului	4
3.1	Descrierea Descompunerii	4
3.2	Descrierea Dependențelor	5
3.3	Descrierea Interfețelor	6
3.3.1	Interfața CLI Client – SOAP Server	6
3.3.2	Interfața Admin Client – Server	6
3.3.3	Interfața Server – Processing Manager	6
3.3.4	Interfața Processing Manager – Worker Processes	6
3.4	Interfețele Modulelor	7
3.5	Interfețe Utilizator	7
3.5.1	CLI Client	7
3.5.2	Admin TUI	7
4	Design Detaliat	9
4.1	Design Detaliat al Modulelor	9
4.1.1	Fluxul Apply Filter	9
4.1.2	Fluxul Monitor și Control Admin	10
4.1.3	Fluxul REPL al CLI Client-ului	11
4.2	Design Detaliat al Datelor	11
4.2.1	Entități Cale SOAP (CLI)	11
4.2.2	Entități Cale Admin (<code>dataTypes.h</code>)	11
4.2.3	Constante Tip Mesaj (<code>dataTypes.h</code>)	12
4.3	RTM (Matrice de Trasabilitate a Cerințelor)	13

1 Introducere

1.1 Scop

Acest Software Design Document (SDD) descrie arhitectura și design-ul sistemului pentru Parallel Image Filtering System (PIF), o aplicație client-server care realizează filtrarea paralelă a imaginilor. Sistemul permite utilizatorilor să încarce o imagine dintr-un terminal CLI, să aplice un filter selectat pe server folosind patru procese worker paralele și să recupereze rezultatul procesat. Un panou de administrator separat, bazat pe terminal, monitorizează și controlează server-ul printr-un UNIX domain socket.

Acest document este destinat managerilor de proiect, inginerilor software și evaluatorilor de proiect care vor fi implicați în implementarea, mentenanța și validarea sistemului.

1.2 Domeniu de Aplicare

Acest document descrie detaliile de implementare ale aplicației PIF. PIF este compus din patru componente principale: SOAP Server, Processing Manager, CLI Client și Admin Client. Fiecare componentă va fi explicată în detaliu în acest Software Design Document.

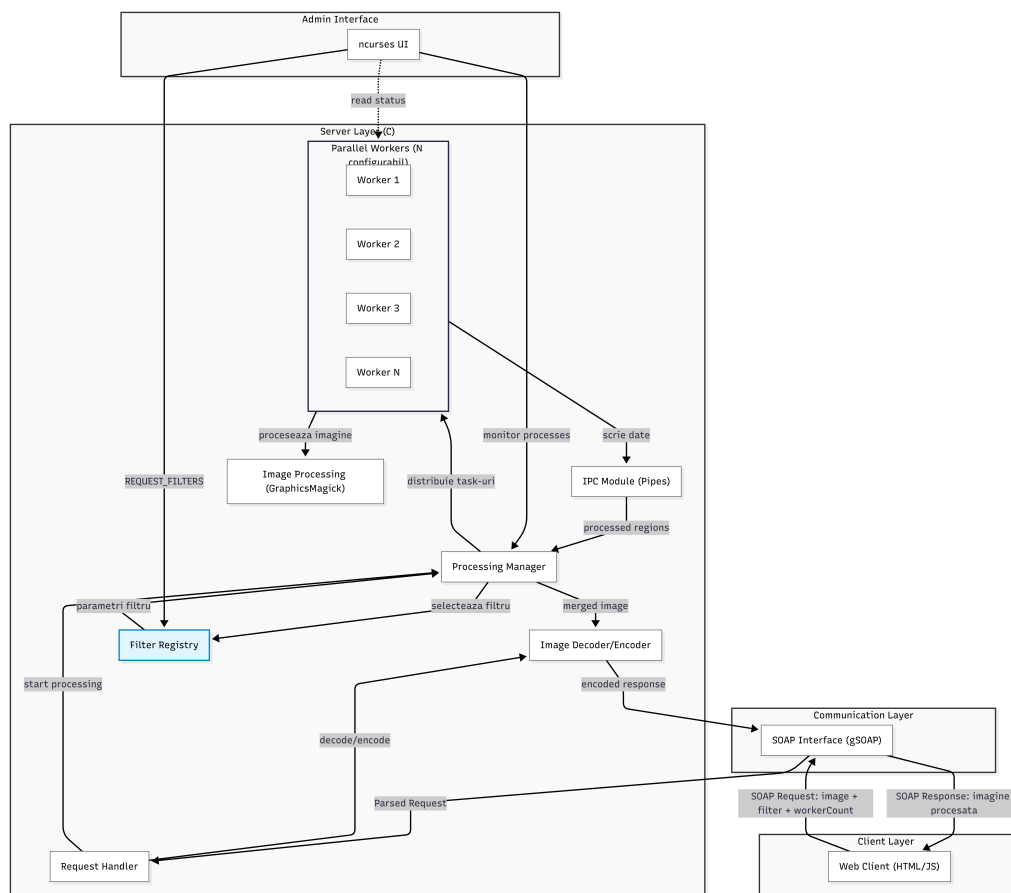
1.3 Definiții, Acronime și Abrevieri

headerblue	
PIF	Parallel Image Filtering
SDD	Software Design Document
SOAP	Simple Object Access Protocol
gSOAP	Toolkit pentru servicii SOAP în C/C++
TCP	Transmission Control Protocol
IPC	Comunicare Inter-Proces
TUI	Interfață Utilizator Terminal
CLI	Interfață Linie de Comandă
REPL	Read-Eval-Print Loop
FIFO	Primul Intrat, Primul Ieșit
API	Interfață de Programare a Aplicațiilor
RTM	Matrice de Trasabilitate a Cerințelor
WSDL	Web Services Description Language

1.4 Referințe

- Documentație gSOAP — <https://www.genivia.com/doc/soapdoc2.html>
- Documentație GraphicsMagick API — <http://www.graphicsmagick.org/api/api.html>
- Documentație libconfig — <https://hyperrealm.github.io/libconfig/>
- ncurses Programming HOWTO — <https://tldp.org/HOWTO/NCURSES-Programming-HOWTO/>
- Gestionare procese POSIX (fork, waitpid, pthread) — IEEE Std 1003.1

2 Prezentare Generală a Sistemului



FIGURĂ 1: Arhitectura de nivel înalt a sistemului PIF

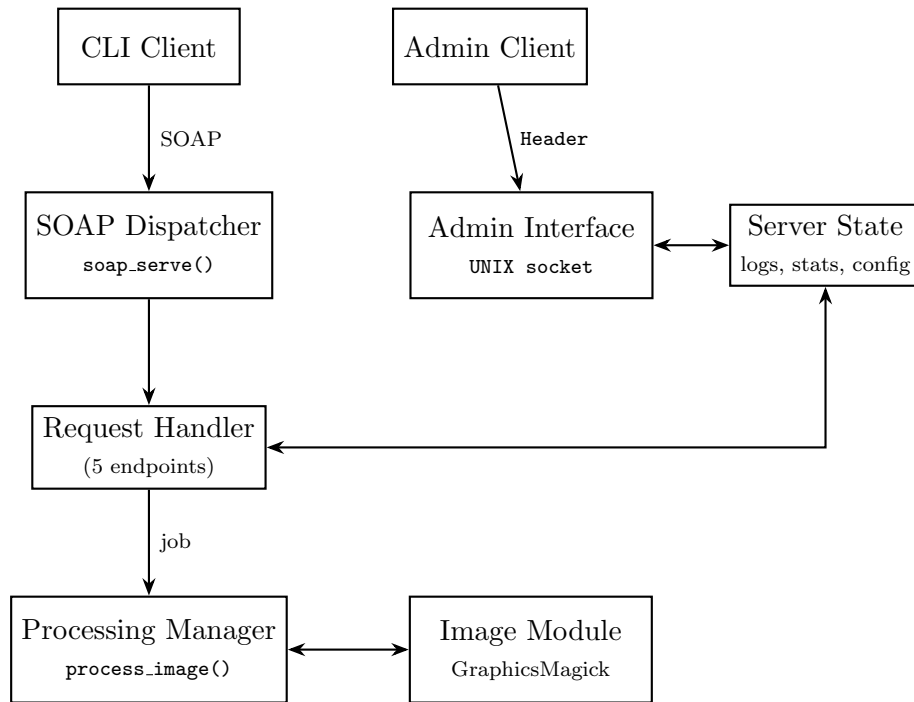
Figura 1 de mai sus reprezintă structura arhitecturală a sistemului PIF. Componenta centrală este PIF Server, scris în C folosind gSOAP și POSIX threads. Server-ul expune o interfață SOAP peste HTTP/TCP pe portul 18082, consumată de CLI client-ul bazat pe terminal. Partea de admin este menținută deliberat separată: administratorul se conectează la server printr-un UNIX domain socket la `/tmp/unixds`, care este mai rapid decât o conexiune TCP localhost și impune permisiuni de acces la nivel de filesystem.

Layer-ul de procesare este invocat intern de server la fiecare request `applyFilter`. Processing Manager-ul (`process_image()`) decodează blob-ul de imagine primit folosind GraphicsMagick, împarte imaginea în patru cadrane egale, fork-ează patru procese worker (câte unul per cadran), colectează rezultatele după ce toți copiii se termină și reassemblează imaginea finală. Folosirea `fork()` în loc de thread-uri oferă izolare per worker: dacă un worker se blochează din cauza unui fault GraphicsMagick, server-ul și ceilalți worker-i rămân neafecți.

În plus, prin urmarea acestei arhitecturi, sistemul poate fi extins în viitor cu o coadă FIFO de job-uri între SOAP thread și Processing Manager, permițând programarea job-urilor în ordinea sosirii fără a modifica interfața SOAP.

3 Componentele Sistemului

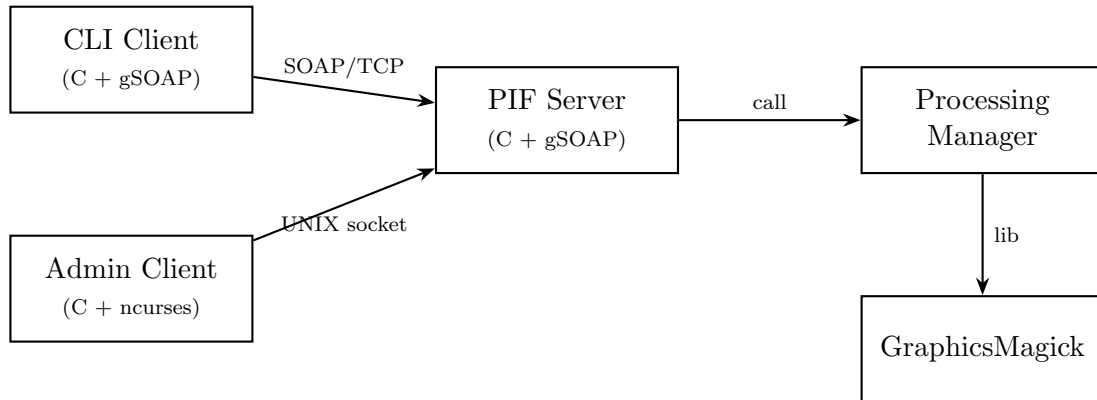
3.1 Descrierea Descompunerii



FIGURĂ 2: Descompunerea top-down a sistemului PIF

Figura 2 de mai sus prezintă o descriere top-down a modului în care aplicația PIF este așteptată să funcționeze și cum interacționează componentele. Cei doi clienți de sus (CLI, Admin) sunt procese independente care comunică cu server-ul prin transport-uri diferite. SOAP Dispatcher-ul primește request-uri HTTP/SOAP de la CLI client și le rutează către Request Handler, care implementează cele cinci endpoint-uri gSOAP (`ns__connect`, `ns__echo`, `ns__applyFilter`, `ns__bye`, `ns__serverInfo`). Admin Interface primește request-uri binary-framed de la Admin Client peste UNIX socket și citește/scrie starea server-ului (log-uri, statistici, configurație). Processing Manager este invocat exclusiv de `ns__applyFilter` și se bazează pe Image Module (GraphicsMagick) pentru a decoda, filtra și re-encode datele imaginii. Toată starea server-ului vizibilă administratorului (contoare de utilizare a filter-elor, ring buffer de log-uri, configurația curentă, clienți conectați) este partajată între Request Handler și Admin Interface.

3.2 Descrierea Dependențelor

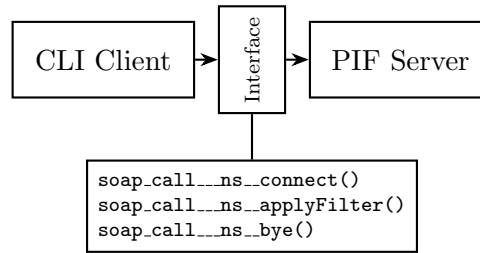


FIGURĂ 3: Diagrama dependențelor componentelor

Figura 3 de mai sus reprezintă diagrama de dependențe a componentelor sistemului PIF și cum fiecare modul depinde de altul pentru funcționalitatea sa. Utilizatorul începe interacțiunea printr-un client (CLI sau Admin). CLI client-ul construiește envelope-uri SOAP și le trimite peste TCP către PIF Server, care le direcționează către unul din cei cinci handler-i de endpoint. Handler-ul pentru `ns_applyFilter` delegă procesarea efectivă a imaginii către Processing Manager, care la rândul său apelează GraphicsMagick prin API-ul său de bibliotecă C. Toți copiii lui `process_image()` (cele patru procese worker) partajează același blob de intrare read-only și scriu rezultatele în fișiere temporare pe care părintele le reasamblează. Admin Client ocolește complet SOAP: deschide un UNIX domain socket către server și schimbă mesaje binare de format fix (**Header** + payload) pentru a interoga starea și a trimite configurații.

3.3 Descrierea Interfețelor

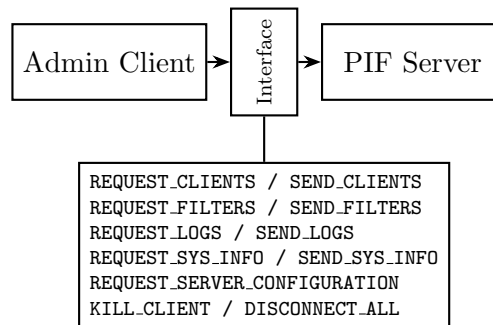
3.3.1 Interfața CLI Client – SOAP Server



FIGURĂ 4: Interfața CLI Client – SOAP Server

CLI Client-ul citește o imagine din filesystem-ul local și folosește stub-uri generate de gSOAP pentru a apela trei endpoint-uri pe server. Imaginea este transmisă în interiorul envelope-ului SOAP ca element `xsd:base64Binary`, pe care gSOAP îl base64-encodează automat pe fir și îl decodează pe partea server-ului.

3.3.2 Interfața Admin Client – Server



FIGURĂ 5: Interfața Admin Client – Server (UNIX socket)

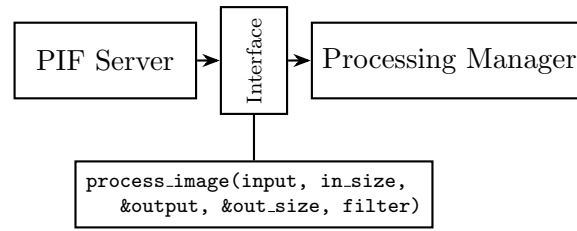
Interfața Admin-Server folosește un protocol binar ușor definit în `dataTypes.h`. Fiecare schimb începe cu un struct `Header` de dimensiune fixă care descrie tipul request-ului și dimensiunea payload-ului, urmat de zero sau mai multe înregistrări ale tipului de struct corespunzător (ex. `ClientInfo`, `Filter`, `LogEntry`, `SysInfo`, `ServerConfiguration`, `toKill`).

3.3.3 Interfața Server – Processing Manager

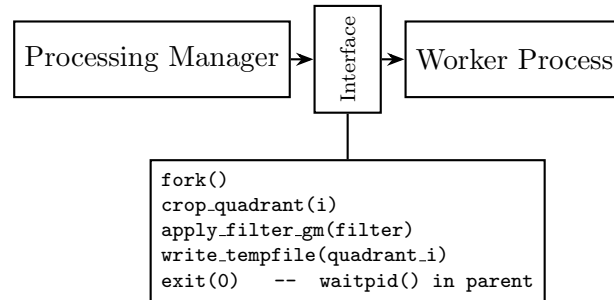
Aceasta este un apel de funcție C în-proces. Server-ul transferă ownership-ul blob-ului de ieșire apelantului, care îl copiază în memoria gestionată de gSOAP (`soap_malloc`) și eliberează buffer-ul original.

3.3.4 Interfața Processing Manager – Worker Processes

Părintele fork-ează patru procese copil, fiecare calculând un cadran al imaginii. Rezultatele sunt returnate prin fișiere temporare ale căror căi sunt deterministe (`/tmp/pif-<pid>-q<i>`). Părintele apelează apoi `waitpid` pentru fiecare copil și recitește cele patru cadrane.



FIGURĂ 6: Interfața Server – Processing Manager



FIGURĂ 7: Interfața Processing Manager – Worker Process

3.4 Interfețele Modulelor

Cele cinci interfețe descrise mai sus sunt singurele granițe cross-modul din sistem. Toate celelalte interacțiuni (ex. bucla REPL care apelează `client_apply_filter()` pe partea CLI) sunt interne unui singur modul și urmează semantica directă a apelului de funcție fără un contract de interfață formal.

3.5 Interfețe Utilizator

3.5.1 CLI Client

CLI client-ul este un terminal REPL. La pornire citește configurația din (în ordine) valori implicite hardcodate, variabile de mediu, `client.cfg` și opțiuni din linia de comandă; se conectează la server și prezintă un prompt `pif>`. Utilizatorul tastează comenzi de forma `<filter> <input> <output>` pentru a aplica un filter, `help` pentru a lista filter-ele și comenzile suportate, și `exit` (sau Ctrl+D) pentru a se deconecta ordonat. La erori de rețea, REPL-ul încearcă o reconectare automată înainte de a eșua.

3.5.2 Admin TUI

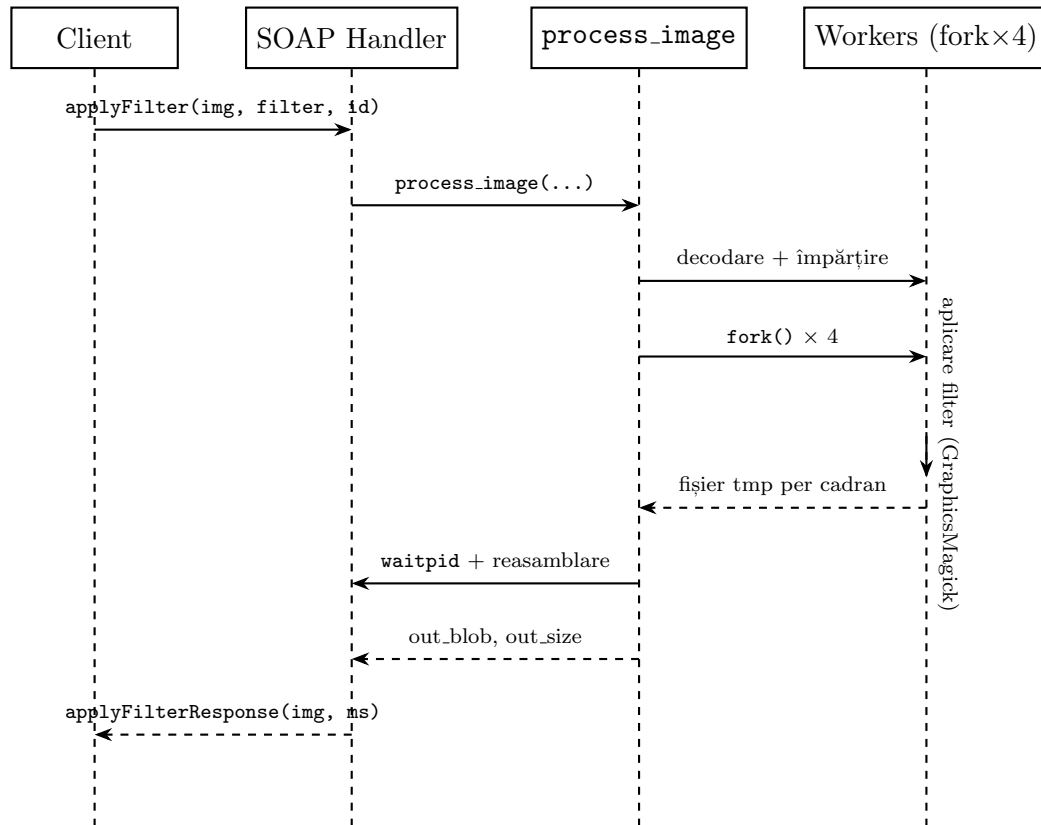
Admin TUI-ul este construit cu ncurses. Meniul principal este o fereastră centrată cu șase intrări: Monitor & Control, Filter Statistics, Configuration, View Logs, Server Usage, Exit. Fiecare intrare deschide un ecran dedicat. Ecranul Monitor & Control afișează un tabel scrollabil al clienților conectați (IP, job ID, patru rânduri de worker per client cu PID, CPU%, RAM%, status) și permite administratorului să termine un client apăsând ENTER pe rândul său, ceea ce deschide un popup de confirmare Yes/No. Filter Statistics afișează un tabel cu numele filter-elor și contoarele de utilizare. Configuration comută starea server-ului OPEN/CLOSED, editează numărul maxim de clienți concurenți cu tastele UP/DOWN și deconectează

toți clienții. View Logs afișează un pad scrollabil cu mesajele recente de log. Server Usage afișează CPU%, RAM folosit/total și string-ul de uptime al server-ului. Pe toate ecranele, tastele săgeată navighează/scrolează, ENTER confirmă și x revine la ecranul anterior.

4 Design Detaliat

4.1 Design Detaliat al Modulelor

4.1.1 Fluxul Apply Filter



FIGURĂ 8: Diagramă de secvență pentru operația Apply Filter

Diagramă de Secvență.

Pseudocod.

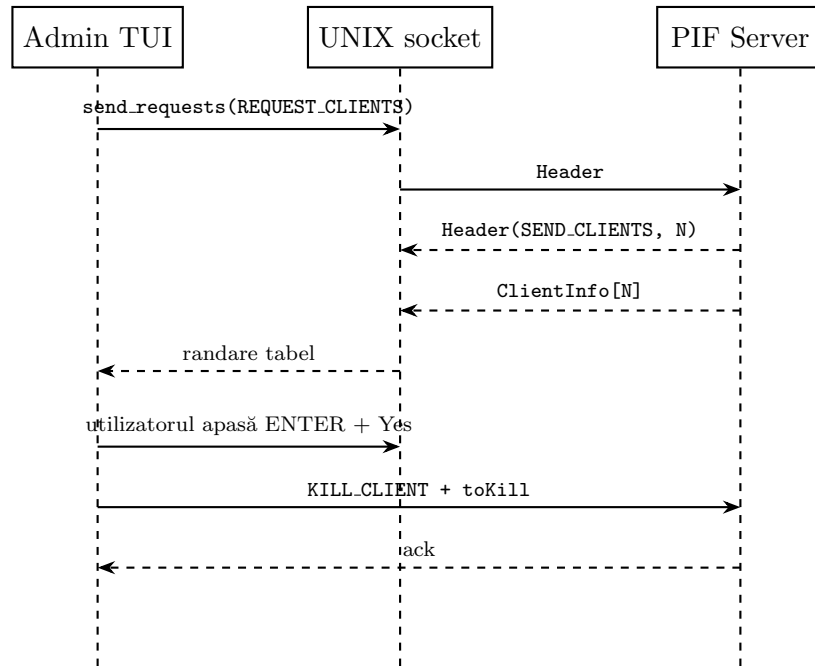
```
Client builds SOAP request with imageData, filterType, clientId
Client POSTs to http://server:18082
gSOAP dispatcher routes to ns__applyFilter()
    record start_time via gettimeofday()
    call process_image(img_ptr, img_size, &out, &out_size, filter)
        decode input blob via MagickReadImageBlob()
        read width W, height H
        for i in 0..3:
            fork():
                crop quadrant i from image
                apply filter (grayscale / blur / sharpen / edge / negative)
                write quadrant to /tmp/pif_<pid>_q<i>
                exit(0)
        for i in 0..3: waitpid()
```

```

    stitch 4 quadrants into final image
    encode to memory blob
    return blob
copy out into soap_malloc buffer
resp.processingTime = now - start_time
return SOAP_OK
Client receives SOAP response, writes output file or renders image

```

4.1.2 Fluxul Monitor și Control Admin



FIGURĂ 9: Diagramă de secvență pentru operația de terminare client admin

Diagramă de Secvență.

Pseudocod.

```

connect_to_server(): socket(AF_UNIX, SOCK_STREAM); connect("/tmp/unixds")
send_requests(fd, REQUEST_CLIENTS)
recv(fd, &header, sizeof(Header))
assert header.type == SEND_CLIENTS
arr = malloc(sizeof(ClientInfo) * header.count)
loop recv() into arr until all bytes received
render ncurses pad with client rows
user navigates with UP/DOWN, presses ENTER on a row
confirmation_window(): user selects Yes
build toKill { job_id, childrens[4] }
send_requests(fd, KILL_CLIENT)
send(fd, &toKill, sizeof(toKill))
recv ack header
remove client from local array, redraw

```

4.1.3 Fluxul REPL al CLI Client-ului

Pseudocod.

```
config_set_defaults(cfg)
config_load_env(cfg)
config_load_file(cfg, "./client.cfg")
config_parse_args(cfg, argc, argv)
soap = soap_new()
client_connect(soap, cfg, &client_id)
loop:
    print "pif> "
    read line from stdin
    if line == "exit" or EOF: break
    if line == "help": print_help(); continue
    parse "<filter> <input> <output>"
    rc = client_apply_filter(soap, cfg, client_id, filter, input, output)
    if rc == -1:
        soap_end(soap)
        client_connect(soap, cfg, &client_id) // auto-reconnect
client_bye(soap, cfg, client_id)
soap_destroy(soap); soap_end(soap); soap_free(soap)
```

4.2 Design Detaliat al Datelor

4.2.1 Entități Cale SOAP (CLI)

headerblue		
imageData	xsd:base64Binary	Bytes imagine (_ptr, _size); encodat automat de gSOAP
filterType	string	grayscale, blur, sharpen, edge, negative
processCount	int	4 (hardcodat în CLI)
clientId	int	Atribuit de ns:connect, limitat la sesiunea TCP
processingTime	int	Ms scurși returnați alături de rezultat

4.2.2 Entități Cale Admin (dataTypes.h)

headerblue	
Header	size_t type, size_t size, int count
ClientInfo	char ip[16], int job_id, ProcessInfo P[4]
ProcessInfo	pid_t pid, int cpu, int ram, char status[8]
Filter	char name[64], unsigned int uses
toKill	int job_id, pid_t childrens[4]
ServerConfiguration	char status[6] (OPEN / CLOSED), unsigned int max_clients_number
LogEntry	char message[256]
SysInfo	int CPU_usage, long RAM_usage, long RAM_max, char uptime[64]

4.2.3 Constante Tip Mesaj (dataTypes.h)

headerblue	
REQUEST_CLIENTS	Solicită lista clienților conectați
SEND_CLIENTS	Răspuns cu lista clienților
REQUEST_FILTERS	Solicită statisticile de utilizare a filter-elor
SEND_FILTERS	Răspuns cu lista filter-elor
KILL_CLIENT	Termină worker-ii unui client specific
DISCONNECT_ALL	Deconectează toți clienții
REQUEST_SERVER_CONFIGURATION	Solicită configurația curentă
SEND_SERVER_CONFIGURATION	Trimite sau primește o configurație
REQUEST_LOGS	Solicită înregistrările de log ale server-ului
SEND_LOGS	Răspuns cu înregistrările de log
REQUEST_SYS_INFO	Solicită statistici CPU, RAM și up-time
SEND_SYS_INFO	Răspuns cu informațiile de sistem

4.3 RTM (Matrice de Trasabilitate a Cerințelor)

headerblue			
3.2.1.2	Încărcare imagine (CLI)	CLI client (client_apply_filter)	1.2
3.2.1.3	Procesare paralelă imagine	ns__applyFilter + process_image + Graphic- sMagick	2.1, 2.2
3.2.1.4	Execuție paralelă $\times 4$	fork() în process_image()	2.3
3.2.2.1	Fișier de configurare (lib- config)	config_load_file() + client.cfg	3.1
3.2.2.2	Configurare prin variabile de mediu	config_load_env() (PIF_*)	3.2
3.2.2.3	Parsare argumente CLI	config_parse_args() via getopt()	3.3
3.2.3.1	Monitorizare admin	Admin Client (ncurses) + UNIX socket	4.1, 4.2
3.2.3.2	Configurare admin	Admin Client + ServerConfiguration	4.3
3.2.3.3	Statistici utilizare filter	Filter struct + SEND_FILTERS	4.4
3.2.3.4	Vizualizare log-uri server	LogEntry[] + SEND_LOGS	4.5
3.2.3.5	Monitorizare resurse server	SysInfo + SEND_SYS_INFO	4.6
3.2.3.6	Terminare client specific	toKill + KILL_CLIENT	4.7
3.2.4.1	Verificare disponibilitate / ping	ns__echo() + SOAP.ping()	5.1
3.2.4.2	Gestionare sesiune client	ns__connect() / ns__bye()	5.2